

Lecture 6

Tuesday February 10th

Quiz

- Tuesday 19th will be the first quiz
- Email professor about not being able to use Lock down browser

Recap

- Talked about processes
- What is inside of a process
- Stack to do function calls and data to be operated on
- Processes have different states for interacting with the OS
- Processes create children in a spawn process
- Went over code examples using the `fork()` command to spawn child processes
- the fork command is exponential meaning if `fork()` is called 3 times in a program you will have 2^3 processes.
- `execvp(myargs[0], myargs)` will flush the program commands and over write the code with something new (whatever is in the myargs)

Process Termination

- events that lead to process termination (exit)
 - regular completion with or without error code
 - the process voluntarily executes an `exit(err)` system call to indicate to the OS that it has finished
 - fatal error (uncatchable or uncaught)
 - service error
 - total time limit exceeded
 - arithmetic error, out-of-bounds memory
 - killed by another process via the kernel
 - process receives a `sigkill` signal

Bonus question discussion

1. the value of x in the child and parent will start with 100. If the child and parent both go on to change the value of the variable, say child subtracts and parent adds 10 to the variable then the variable will be different in both processes

2. With the open system call in the parent and then calling fork both child and parent will be able to open the file. If they write to the file concurrently it will cause problems with over-writing each other. This can be handled with locks and synchronization

Program Execution

- list of things that occur when a program starts
- an entry is created in the process list
- allocate memory
- load program into memory
- set up the stack with argc/argv
- clear registers
- execute call main()
- Free memory of process
- Remove from process list

Restricted Operation

- You can run anything on your computer
- write to the whole disk
- wipe out program memory
- Applications must be prevented from doing restricted operations:
 - system calls to the OS
 - `fork()`
 - `write()`
 - `open()`
 - `wait()`
- Need to have strict permissions for applications to these privileges

User Mode vs Kernel Mode

- all apps run in user mode
 - less privileged
 - code cannot do restricted operations
 - In user mode, app cannot issue I/O
 - kernel mode
 - OS runs
 - highly privileged
 - code can do almost anything in this mode

- look at slide showing the system calls

Process Pause / Dispatch

- Some events that lead to process pause or dispatch
 - I/O wait
 - Preemptive timeout

Process "Context" switching

- when CPU switches to another process, the system must save the state of the old process and load the saved state for the new process
- Context-switch time is overhead; the system does no useful work while switching
- Switching time is dependent on hardware support
- Air B&B example
- Process needs to maintain multiple queues with a table
 - See slide for the representation of the process scheduling